PES1UG24AM126

KR Tanishkaa

Step 1: Scenario & Requirements Breakdown

Before designing, we extract the core elements from the scenario:

- **Core Functionality:** Select 3 coffee types (Espresso, Americano, Latte), choose from 2 sizes (Small, Large), accept credit card payments only, print order receipt.
- **Technical Constraints:** Touchscreen UI hardware, local receipt printer hardware, menu/pricing storage.
- **Key Challenges:** Fast order throughput (performance), immediate user feedback (usability), secure credit card processing (PCI compliance/security).

Step 2: Architectural Style Analysis

Architectural Style	Evaluation for the Self-Service Kiosk Scenario
Layered	**Best Fit.** Clear separation between UI, business logic, and local hardware/database integrations. Simple to deploy on single standalone physical kiosks.
Microservices	Over-engineered for a standalone physical kiosk. Adds network latency, operational overhead, and unnecessary deployment complexity.
Client-Server	Suitable if managing hundreds of networked kiosks, but adds single points of failure if remote server connection drops during ordering.

Step 3: Component Identification & Interface Mapping

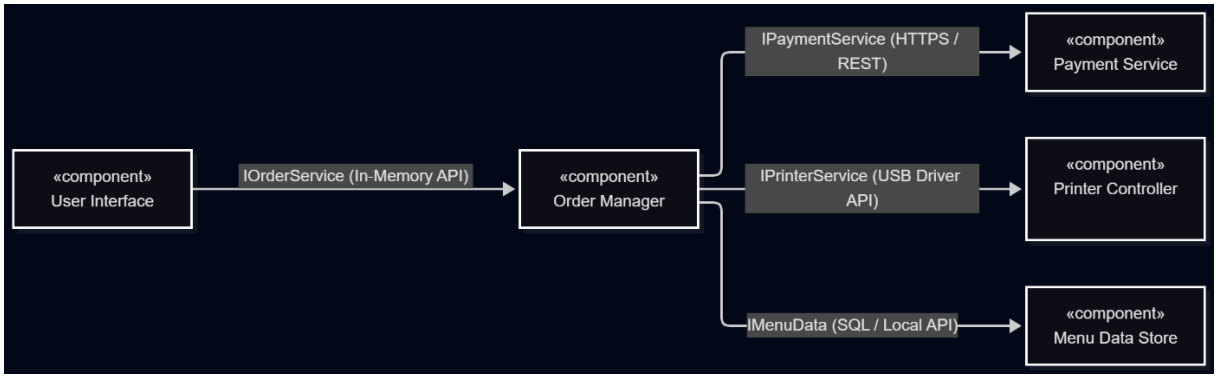
We need **5 components** and at least **4 interfaces**:

The 5 Components:

1. **User Interface Component:** Handles user touches and displays the coffee menu/order status.

2. **Order Manager Component:** Core business logic for processing orders and item customization.
3. **Payment Service Component:** Handles credit card validation and payment processing.
4. **Printer Controller Component:** Interface driver to communicate with the receipt printer hardware.
5. **Menu Data Store Component:** Stores/retrieves menu items, pricing, and sizing details.

Interface Name	**Consumer (Required / Socket)**	**Provider (Provided / Ball)**	**Protocol / Communication Type**
`IOrderService`	User Interface	Order Manager	In-Memory API Call / Event Handler
`IPaymentService`	Order Manager	Payment Service	HTTPS / REST API (Secure Tokenization)
`IPrinterService`	Order Manager	Printer Controller	USB / Serial Driver API
`IMenuData`	Order Manager	Menu Data Store	Local SQL / Data Access API



Architectural Selection & Justification: Self-Service Coffee Kiosk System

Architecture Selection

We chose **Layered Architecture** for the Self-Service Coffee Kiosk System.

Concrete Justification

1. **Separation of Hardware and Business Logic:** The kiosk relies on specific hardware interfaces (touchscreen and receipt printer). A layered pattern cleanly isolates hardware-driver logic (Presentation/Hardware Layer) from core business logic (Order Processing) and data storage, ensuring hardware driver updates do not break ordering logic.
2. **Simplicity and Standalone Reliability:** Because a self-service coffee kiosk functions primarily as a single, standalone hardware unit, a layered monolithic architecture avoids the unnecessary network overhead, deployment complexity, and distributed failures of microservices.

Security Advantage

PCI-DSS Isolation: By isolating the `Payment Service` into a dedicated component boundary within the architecture, credit card data handling is sandboxed. Sensitive payment interactions can be encrypted locally and transmitted via secure tokenization without exposure to the general UI or storage components.

Performance Benefit

Low Latency & Fast Execution: Local, in-memory function calls between the UI, Order Manager, and local Menu Data Store minimize request execution time, ensuring sub-second touch response times for customers during high-volume café hours.